

State Joining and Splitting for the Symbolic Execution of Binaries

Trevor Hansen, Peter Schachte and Harald S ndergaard

Department of Computer Science and Software Engineering
The University of Melbourne, Vic. 3010, Australia

Abstract. Symbolic execution can be used to explore the possible run-time states of a program. It makes use of a concept of “state” where a variable’s value has been replaced by an expression that gives the value as a function of program input. Additionally, a state can be equipped with a summary of control-flow history: a “path constraint” keeps track of the class of inputs that would have caused the same flow of control. But even simple programs can have trillions of paths, so a path-by-path analysis is impractical. We investigate a “state joining” approach to making symbolic execution more practical and describe the challenges of applying state joining to the analysis of unmodified Linux x86 executables. The results so far are mixed, with good results for some code. On other examples, state joining produces cumbersome constraints that are more expensive to solve than those generated by normal symbolic execution.

1 Introduction

Automated software verification aims to ensure that code behaves as intended. Research into the mechanisation of the task began more than 30 ago, but waned as the difficulty of the task became clearer; in particular scalability turned out to be elusive. But recent advances in constraint solving technology have rekindled optimism. At the same time the desire for reliable verification of software, in particular of binary code, is growing.

We are currently developing a tool for the analysis of unmodified Linux x86 executables, for the purpose of identifying violated assertions. The solution requires dynamic disassembly of a given executable. A natural approach is to use symbolic execution on semi-random input to explore the space of possible run-time states.

Symbolic execution derives formulae that precisely describe the output state of a program in terms of its inputs. Symbolic execution has been used to generate test cases [5], and to show the equivalence of programs [10]. In each case, the tremendous number of execution paths limits the size of programs that can be symbolically executed. Although relevant to other uses of symbolic execution, we are interested in state joining in the context of the verification of binary executables. Dealing with the path explosion is the key challenge in making symbolic execution useful for software verification.

By combining the results of symbolic execution along all the program’s paths, the behaviour of the program is described. Although the number of paths through a deterministic program is less than the number of its input states, real programs still have intractably many possible paths. In traditional symbolic execution, states are split at control flow instructions, and not subsequently joined, even where the program’s control flow merges. State joining addresses the path explosion problem by coalescing states with the same instruction pointer.

Dynamic analysis, symbolic execution, and abstract interpretation are three approaches that abstract the concrete semantics of a language differently. Dynamic analysis uses actual values—an under-approximation, symbolic execution uses the complete semantics—no approximation, and abstract interpretation uses an over-approximation. Each establishes the relationship between the input and output. Verification checks that properties apply to this relationship, or sequence of relationships. Each approach can be used to verify just some of the states in a program, partially verifying it, or to verify the entire program.

Complete dynamic analysis, and symbolic execution, check every feasible state, and no unfeasible states. Abstract interpretation checks a superset of feasible states, that is, all feasible and some unfeasible states. Consequently the guarantee from each approach is different. Dynamic analysis misses defects that do not occur for the particular *states* it checks. Symbolic execution misses defects that do not occur for the particular *paths* it checks. Abstract interpretation checks an over-approximation of the states, so perhaps raises false alarms. Symbolic execution and dynamic analysis do not approximate, so find all the defects and no extras—at the cost of tractability. The partial dynamic analysis and the complete abstract interpretation of large programs are common, but the complete symbolic execution of large programs is not.

There are other differences between the approaches. Abstract interpretation is usually designed to terminate, possibly through the use of widening. However, dynamic analysis and symbolic execution will not terminate if the program they are analysing does not terminate. Symbolic execution and dynamic analysis have an advantage that they are able to produce the concrete inputs that cause failures—which allows programmers to reproduce the failure.

We use dynamic disassembly to incrementally build a partial Control Flow Graph (CFG). It is partial, because other blocks or transitions might have occurred if the results from system calls were different, or a different length input was used. We use the partial CFG to determine when states can be joined. In practice having a partial rather than a complete CFG causes minor complications.

The rest of this paper proceeds as follows. Section 2 gives an example to clarify our approach. The example shows the advantages of state joining, disregarding the practical difficulties (these are discussed in Sect. 7). Section 3 contains a description of the overall algorithm. Without applying simplifications the symbolic formulae quickly grow unmanageable. Section 4 describes how Boolean minimisation can be used to reduce a formula’s size. Section 5 describes the specifics of our tool. In Sect. 6 we compare runs with and without state joining.

```

1 uint64_t multiply(uint64_t x0, uint64_t y0) {
2     uint64_t x=x0, y=y0, z=0;
3     while(x!=0) {
4         if (!even(x)) /* (x&1) */
5             z += y;
6         x  = x >> 1;
7         y  = y << 1;
8     }
9     return z;
10 }

```

Fig. 1: Using shift-and-add to multiply positive integers x0 and y0

Mostly state joining helps, but it does produce more cumbersome constraint expressions, which can slow down the analysis. Finally we compare our approach to others that tackle symbolic execution’s path explosion problem.

2 State Joining: An Example

The terminology of symbolic execution (or symbolic simulation) varies, so let us fix our use of terms. We denote the instruction pointer register (which holds the address of the next instruction to execute) by *IP*. A *path* is a sequence of instructions. For a given input, a path can be constructed by running the program and listing the successive IP values. (We assume, without loss of generality, that a program has a single entry point, $IP_{initial}$, and a single exit point IP_{final} .) When instructions are in loops, or called multiple times in functions, those instructions will occur multiple times in the path. By a *trace* we mean a path and the input and output of the system calls. A (symbolic) *state* is a map from locations to expressions, together with the IP, where a *location* may be a register or a memory address. With each state we shall also associate a “path constraint” (see below). A state is *feasible* if it can occur along some path.

Symbolic execution may *fork* at some conditional branch instructions. It is natural to associate a *path constraint* (PC) with a state, the PC being a constraint over the program input variables. The PC describes the inputs that would take the same path through the program, that is, the inputs for which the associated state is valid. States can be seen to form a *state tree*, with branching occurring whenever a conditional branch instruction depends on symbolic values. In the state tree, a parent’s path constraint is equivalent to the disjunction of its children’s path constraints. In the presence of “state joining,” the tree becomes a (directed, acyclic) *state graph*.

To provide the intuition of state joining we will describe the analysis of the Multiply example in Fig. 1. The program uses shift-and-add to multiply two non-negative integers x and y , leaving the result in z . We show C-style code for clarity—all analysis is performed on unmodified executables.

Figure 2 shows a control-flow graph for the function and Fig. 3 shows the state graph that is produced during joining. For each distinct x value, a Multiply call takes a different path through the function body; these many paths make symbolic execution slow. For 64-bit integers, there are 2^{64} distinct paths through the function. To see this consider the branching on the even condition (block B_3), which checks the rightmost bit of the x variable. Then the bits are shifted one to the right, giving a new rightmost bit. For a given x_0 , the resulting symbolic expression for z is an expression valid for every y value. For example, given $x_0 = 2$, the symbolic expression produced for z is $z = (y_0 \ll 1)$, with a PC that simplifies to $x_0 = 2$. The resulting expression for z describes the output state in terms of the input state. Substituting the input values into the formula for z gives exactly the same result as if the function was run with the same inputs.

Symbolic execution addresses the state explosion problem by reasoning about all the states that take a particular path. In this case there are 2^{64} paths to symbolically execute, versus $2^{64} \times 2^{64}$ initial states. We now face the path explosion problem—this function still contains 2^{64} paths, too many to reason about one-by-one. With state joining, states with the same IP are joined. This makes good sense for the example, as two paths that have previously differed in whether one bit was one or zero could have identical subsequent paths.

Each time a control transfer instruction is reached, a constraint solver call is made to check which branches can be taken. For example, on reaching block B_2 , in Fig. 2, we call the solver to check which of the branches to points 2 or 5 can be taken. In this example both branches can be taken, so the state is split, with one branch conjoining the PC with $x = 0$, and the other using $x \neq 0$. We may consider the state of the function to have 5 elements: x , y , z , the block that will next be executed (the IP), and the PC. Figure 3 shows how the states are split and joined. The topmost state in the figure corresponds to having just entered the function, with the IP at program point 1. The second row results from handling the $x == 0$ condition. One state

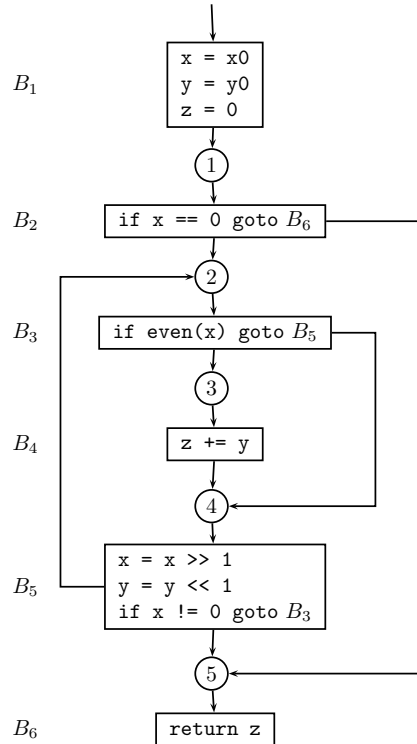


Fig. 2: CFG for multiplication code

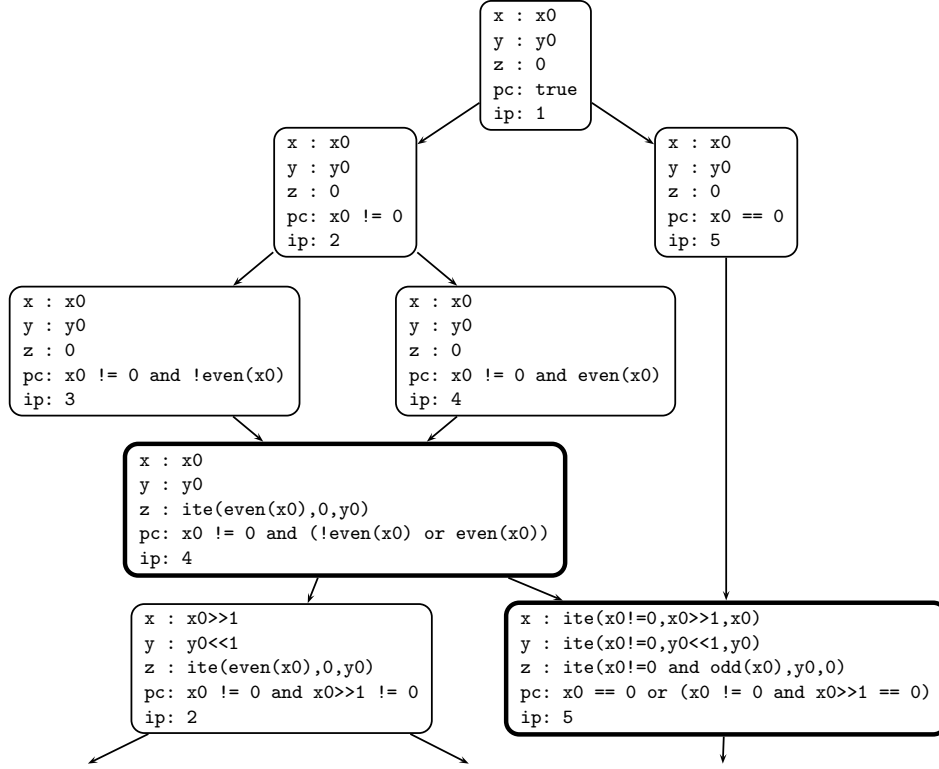


Fig. 3: The state graph for the example of Fig. 2.

corresponds to $x == 0$, the other to entering the loop. There are two joins, shown with bold borders. If the states being joined have the same expression, then the value is unchanged, but if they are different, an ITE constructor (for if-then-else) joins the different expressions. The result of a join holds all the information of the component states. No information is discarded.

Eventually x 's value at the end of B_5 is $(x0 >>_u 64)$, so that $x \neq 0$ is unsatisfiable. Then only the branch to point 5 will be taken. When this happens, the final states will be joined together at point 5, producing the complete expression for z in terms of its inputs. The loops are unrolled automatically—no domain knowledge needs to be entered about the number of times to unroll loops. Note that this analysis is “anytime”: If the analysis is paused, information can be read out that correctly describes the program's runtime behaviour.

3 The Algorithm

The algorithm (Algorithm 1) takes a program to analyse, in the form of a CFG, and the initial state comprising the PC true, the IP $IP_{initial}$, and symbolic variables for all inputs. The symbolic execution will then produce results covering all possible values of these symbolic variables. The algorithm presented is somewhat

Algorithm 1 Applying state joining to a program

Require: A *CFG* and an initial symbolic state s_0

```
1:  $Fringe \leftarrow \{s_0\}$ 
2: while some  $s \in Fringe$  has  $ip(s) \neq IP_{final}$  do
3:   for all  $\{s_1, s_2\} \subseteq Fringe$  such that  $ip(s_1) = ip(s_2)$  do
4:      $Fringe \leftarrow \{s_1 \sqcup s_2\} \cup (Fringe \setminus \{s_1, s_2\})$ 
5:   end for
6:    $s \leftarrow \text{choose}(Fringe)$ 
7:    $Fringe \leftarrow (Fringe \setminus \{s\}) \cup \text{execute}(s)$ 
8: end while
9: return  $s$  where  $\{s\} = Fringe$ 
```

simplified, in that it does not show the decompilation of the program needed because we are analysing binaries. In practice, we interleave decompilation with the symbolic execution.

Algorithm 1 relies on three key functions not defined here: `execute`, `choose`, and $\sqcup$ (join on states). The `execute` operation extends the supplied state by executing the instruction at its IP. Note that this will produce more than one state for selection (conditional or computed branch) instructions. The next sections describe the `choose` and $\sqcup$ functions.

3.1 Preparing to Join

We want to propagate states then stop them where they can be joined. Figure 4 shows two fragments of control-flow graphs. On the left, state m 's instruction pointer is two blocks away from a join point, while state n 's instruction pointer is one block away from the same point. We wish to execute n for one block, and m for two blocks. The two states could then be joined. If either state is run too far, past the same-IP point, that chance to join states is lost.

For each state we find all of the descendants of that state in the partial CFG. Because every path finishes at the same exit block, the paths from the states will intersect. For each state we find the minimum distance of that state to its earliest descendant that is common to another state—we call these join points. We find for each state the minimum number of edges that can be traversed before a join point is reached. For each state we now have the minimum distance to its next join point.

Next, we remove from consideration as the next state to run, any state that post-dominates another state. A node o post-dominates another node p if all paths from p to the exit node must pass through o . Since the post-dominated node p should pass through the dominator o , we do not wish to execute the post-dominating state.

The right of Fig. 4 shows an example where the post-dominance check applies. Each node is zero distance from its earliest join point. That is, if each node is not advanced, another state may be joined with it. In the figure if state o were advanced, it would be moved away from its join with states q and p .

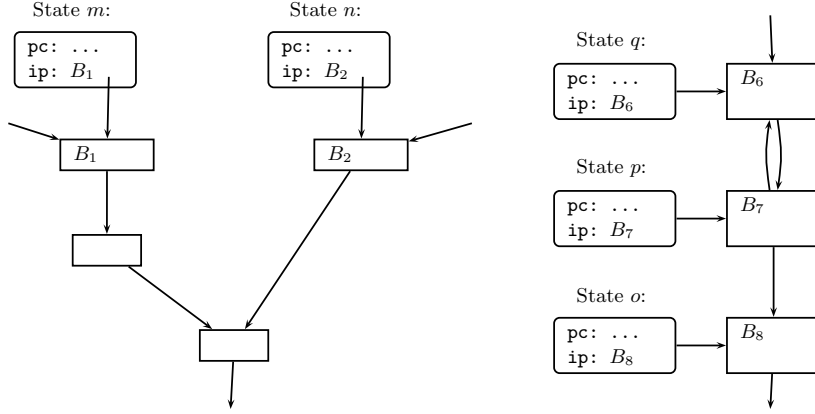


Fig. 4: CFG fragments. State m 's IP is two blocks from a join point. State n 's IP is one block away.

We then choose a state to run for as many blocks as it is from its nearest join point. This is not perfect. We build the control flow graph from all the control flow transfers that have occurred so far in the simulator, as well as the static jumps—that is, jumps to a fixed location, where that location has already been disassembled. Runtime calculated jumps (such as returns from functions) which we have not yet seen are omitted. When control transfers to a new location, we perform a dynamic disassembly and incorporate the resulting blocks into the control flow graph. Note that, if the nearest join point is missed, this does not compromise correctness.

3.2 Joining and Splitting

States are joined, and sometimes split. A join occurs when two states will execute the same instruction next. A split occurs when the location of the next address to execute depends on a symbolic expression. We split in two contexts. Firstly, on reaching a conditional branch instruction whose condition φ depends on symbolic values, the current PC is conjoined with φ , and separately with $\neg\varphi$. If both are satisfiable, the state is split and two states are created.

The second context involves states that have previously been joined. When a function call may return to multiple locations because the return address was joined from states that called the function at different locations, the state needs to be split at the **return** statement of the function to return the respective states back to their call sites. We create a new state for each distinct next instruction. For example, if the instruction pointer can be 2 or 4, then we split the state, resulting in two states, one with a PC of $(PC \wedge IP = 2)$ and another with a PC of $(PC \wedge IP = 4)$.

To join states s_1 and s_2 where s_k contains PC_k and register and memory locations $loc[i]$, create a new state s with $PC_s = PC_1 \vee PC_2$, and for all i ,

$loc_s[i] = ITE(PC_1, loc_1[i], loc_2[i])$. That is, if PC_1 is true, use the value from the first state, otherwise use the value from the second state. This is allowable because PCs are always disjoint.

4 Simplifications and Approximations

Symbolic execution, even of small programs, can result in large symbolic expressions. This is especially so when analysing machine code. We now discuss some of the simplifications we use. We also apply range and domain analysis of pointers to reduce the number of solver calls. The approximations we use over-approximate the encoding of the constraints, but those constraints describe precisely (without approximation) the behaviour of the program.

4.1 PC Simplification

The Multiply example has a simple branching structure, making its PC easy to simplify. However, programs with more complicated control flow, emanating, for example, from **break** statements, can benefit from PC simplification. Without simplification the PC becomes large.

In this section we simplify the PC by abstracting its primitive constraints as propositional variables. We then apply Boolean simplifications to reduce the number of propositional variables in the PC, hopefully making the PC easier to handle for a theory solver. We use a , b , and c to refer to propositional variables that describe individual $SMT(QF_AUFBV)$ ¹ constraints.

Continuing with the Multiply example, consider point 4, where the $(x \neq 0 \wedge even(x))$ state joins with the $(x \neq 0 \wedge \neg even(x))$ state. Letting $a = (x \neq 0)$, and $b = even(x)$, the joined PC becomes $(a \wedge b) \vee (a \wedge \neg b)$. Applying the obvious simplification reduces the joined state's PC to $(x \neq 0)$. Heuristic DNF minimisation tools that apply such rules are available. We use Espresso [14].

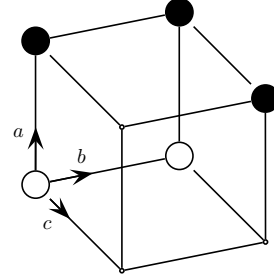
State splitting complicates this minimisation. Consider for example a **return** statement from a function that returns to one of three addresses; perhaps because calls from three different sites were joined. Let the potential new IP addresses be 4, 8 and 12, let the PC before the split be PC_0 , and let IP be the symbolic expression for the instruction pointer when the transfer occurs. Then after the split there will be three states: $(PC_0 \wedge IP = 4)$, $(PC_0 \wedge IP = 8)$ and $(PC_0 \wedge IP = 12)$. If all three states are later joined, the PC will become $PC_0 \wedge (IP = 4 \vee IP = 8 \vee IP = 12)$ which should be simplified to PC_0 . A Boolean minimisation algorithm will not do so—because the second disjunction is not obviously true. We can, however, assist the minimisation algorithm by modifying the constraints. Let $a = (IP = 4 \vee IP = 8)$, and $b = (IP = 4)$. Three equivalent PCs that can easily be minimised are: $(PC_0 \wedge a \wedge b)$, $(PC_0 \wedge a \wedge \neg b)$, $(PC_0 \wedge \neg a)$.

¹ QF_AUFBV is quantifier free formulae over the theory of uninterpreted functions, arrays, and fixed-size bit-vectors. We use the last two. Decision procedures for the theory can solve non-linear formulae with modular arithmetic.

Removing tautologies from the PC helps simplification. For example, consider a PC that contains both $x = 0$, and $\neg(x \neq 0)$. Label them a and b . No state will have the PC $(a \wedge \neg b)$ that can simplify the b term. It is safe to remove constraints with an unsatisfiable negation. Since their negation is unsatisfiable, these constraints subsume prior constraints—they add no information.

State joining produces many ITE expressions, so we carefully simplify their guards. We calculate four possible guards and use the smallest one.

Consider two states to be joined, with $PC_1 = (\neg a \wedge \neg c)$ and $PC_2 = ((a \wedge \neg c) \vee (a \wedge b \wedge c))$. The PCs are shown on the unit cube to the right—hollow circles for PC_1 and filled circles for PC_2 . A reasonable ITE to use for the joined locations is: $loc_{new}[i] = ITE((\neg a \wedge \neg c), loc_1[i], loc_2[i])$. Another reasonable choice for an ITE is to take PC_2 as the guard, and swap the order of the remaining arguments. However it may be possible to generate a smaller guard by considering that the ITE expression will only be evaluated if $PC_1 \vee PC_2$ is true. Inspection of the cube shows that the potentially simpler guard of a is equivalent to $a \wedge \neg c$. a covers only vertices of PC_1 and those that we don't care about, it covers none of PC_2 's vertices.



To minimise the guard, we mark the vertices of PC_1 as true, those of PC_2 as false, and the rest as don't care. Then we minimise using Espresso. Then we swap, marking PC_2 's vertices true, PC_1 's false, and we minimise again.

As guard we choose the expression with the smallest number of nodes in its SMT(QF_AUFBV) representation. These are candidate guards: PC_1 , PC_2 , the restriction of PC_1 to $PC_1 \vee PC_2$, and the restriction of PC_2 to $PC_1 \vee PC_2$.

During the constructions of symbolic expressions we follow standard practice and apply rewriting rules to simplify expressions, for example turning $x + 0$ into x . We implement dozens of simplification rules like some solvers such as STP do. Some SMT(QF_AUFBV) solvers, like Boolector and STP [6], provide an interface to simplify single expressions. Using those interfaces to simplify expression would most likely speed up our tool.

4.2 Value Analysis for Pointers

In the `atol()` function used by a later example (Fig. 5), each character is looked up in an array to determine if it is a digit. So there is a lookup: `isDigit[c]`, where `isDigit` is an array of 256 values indicating whether each c is a digit, or not. We analyse such pointer accesses using three techniques. First by analysing the domain of the expression, if that fails, by analysing the range of the expression, if that fails, by solving for each memory address in turn.

The `isDigit[c]` expression will translate into a symbolic memory access such as `base + (c << 1)`, that is, access the memory location at the value of the character times two plus some fixed base (the location in memory of the zero

index). As this expression contains only a single 8-bit variable, it can take on only 256 possible values. We substitute for each possible value, ignoring the PC.

If the domain of an expression is large, we use a bit-vector interval analysis [13] of the expression to calculate the interval of the range. Balakrishnan [3] gives precise ranges for logical and arithmetic operations. For example a subtract expression over two 8-bit values each with a range of $[0,1]$ has a resulting range of $[-1,1]$. We use a signed range and widen to the whole range when an overflow occurs. A signed range allows us to handle an expression such as $2 * val - 1$. If val is in $[0,12]$, then the range of the expression signed is $[-1,23]$; an unsigned range would widen terribly to: $[1,255]$. Because of the ubiquity of aligned memory accesses we use strides, allowing regular gaps in the interval, for example a stride of 2 on the interval $[0,6]$ contains $[0,2,4,6]$. When performing the interval analysis, we likewise ignore the PC.

If the domain and range analysis produce too many values, or if some of the values are not addressable, we (inefficiently) solve iteratively for the index subject to the PC. For example, given a symbolic expression *index*, which first solves to 2, we then solve *index* such that $index \neq 2$, and so on. The solvers we use do not have the option to produce all the satisfying assignments to a formula—which we require to check all possible memory addresses.

5 The Analysis Tool

We analyse Linux x86 executables, both compiled binaries and interpreted scripts. Given an executable, and an input width, we dynamically disassemble the executable as it runs on a random input of specified length. The result of this dynamic disassembly is a trace through the program. The tool then symbolically executes the program on the same input, splitting states where necessary. When symbolic execution reaches an address that has not already been disassembled, a new dynamic disassembly is triggered. The resulting trace is merged with previous disassembled traces. We follow Minato [10] and check the satisfiability of each guard. A loop is unrolled until its guard evaluates to false.

We use the Valgrind framework [12] to disassemble. Valgrind is a widely used dynamic instrumentation framework for analysing x86 and PowerPC executables on the Linux and AIX operating systems. We use Valgrind to identify instructions, and also because it translates x86 instructions into a more manageable RISC language. There are three advantages of dynamic disassembly over static disassembly. First, during disassembly the results from system calls are collected; these are later played back to the simulated program. Second, we disassemble the dynamically linked libraries no differently from the program’s instructions. Third, interpreted scripts, and their interpreters, can be analysed. Our dynamic disassembly is time consuming, others [11] have used an initial static disassembly to save time.

The PC and symbolic expressions are stored inside our tool as Valgrind expression graphs, and converted to the SMT(QF_AUFBV) language for solving. We do not utilise the solver’s custom interface. Instead we generate standard

```

1 int main(void) {
2     char s[9];
3     long number;
4
5     printf("Enter a Number:");
6     fgets(s, sizeof(s), stdin);
7     number = atol(s);
8     if (number == 12345678) {
9         fail();
10    }
11    return 0;

```

Fig. 5: Number: Error on input 12345678

SMT-LIB format and communicate via temporary files. Using the standard interface requires the solver to redo work but allows easy experimentation with different solvers. SMT(QF_AUFBV) corresponds closely to the low-level machine integer semantics. It handles precisely the non-linear operations we require.

We have built a simulator of Valgrind’s instructions, which executes the instructions symbolically and concretely. It simulates many, but not all, of Valgrind’s instructions. Currently, our tool does not handle multi-threaded programs, asynchronous signals, or floating point instructions.

The program’s state is modified by Linux system calls. We have symbolic versions of some system calls only, the remainder we replay from the traces captured during dynamic disassembly.

6 Results

To explore the usefulness of the state joining approach we analyse four programs. One is a simple C program that will fail if a particular integer is input (Fig. 5). The function describing the result of `atol()` is surprisingly complicated: Given 9 characters of input, there are about 10^{20} different inputs that will evaluate to 1, corresponding to white space followed by an optional plus sign, followed by 1, followed by non-numeric characters. The second is the gzip file compression program. The third is a program that counts the number of 1-bits in a number (Fig. 6). The fourth is the Multiply example of Fig. 1.

In the Number example, an input that causes the program to fail is derived. For the remaining three programs the input-output function is derived. For the Multiply example, this could be used to show the commutativity of x and y . For Wegner this could be used to determine that the return value is always less than 65. For gzip this could be used as the first part of establishing that gunzip composed with gzip is the identity function for some size inputs.

Table 2 shows the results of running these four programs. Some of the rows are: *Dynamic Disassemblies*: the number of calls to the dynamic disassembler

```

1 uint64_t popCount (uint64_t y) {
2     uint64_t c;
3     for (c = 0; y; c++)
4         y &= y-1;
5
6     return c;
7 }

```

Fig.6: Wegner: Counting 1-bits

Example	Bytes	Symbolic Execution	State Joining	Execution
Number	9	3	1	$> 10^5$
Wegner	8	1	1	$> 10^5$
Gzip	1	572	> 4300	1
Multiply	16	$> 10^5$	1	$> 10^5$

Table 1: Normalised times, the fastest approach is given a time of 1.

(the value varies depending on the initial random input chosen); *Joins*: the number of pairs of states that were joined; *Maximum Width*: the greatest number of active states at any time (the maximum width of the state graph); *Maximum Height*: the longest path through the state graph.

We made three runs of each analysis, the times shown in Table 2 are arithmetic averages. Times were measured on a single core of a Pentium D 3GHz, running Ubuntu 8.04. We use revision 60 of STP, which we found to be the fastest available solver.

A program that generates all one byte files, then gzips them, then builds an input-output function takes 7 seconds to run. To produce the equivalent formula by symbolic execution takes 4006 seconds, and to produce the same formula by state joining timed out after 30,000 seconds.

Gzip, when symbolically executed, produces singleton states—each input follows a different path. So symbolic execution has no advantage over dynamic analysis. The PC that state joining produces is more difficult to solve, overwhelming the savings from merging.

The Wegner example has a number of paths equal to the bitwidth plus one, in our example, 65 paths. State joining on this example joins just before the exit—the same as symbolic execution. The Wegner example benefits neither from the Boolean simplifications nor from the pointer value analysis. The Boolean simplifications occur at the join point at the function’s **return**, when the PC is no longer used. The pointer value analysis, as for the Multiply example, does not help because the analysis produces no symbolic pointers.

The Multiply example has a simple structure, with control transfer instructions that join back on each other—producing constraints that easily cancel out.

Category	Problems			
	Number	Gzip	Wegner	Multiply
Bytes input	9	1	8	16
Dynamic Disassemblies	9-16	1	1	1
With joining				
Total time	126s	> 30000s	33s	45s
Solver time (STP v60)	90s	> 26973s	28s	11s
Solver calls	110	> 684	67	131
Boolean simplification	4.5s	> 103s	3s	5.5s
Joins	60	> 510	64	127
Maximum height	94	> 500	67	131
Maximum width	5	2	2	3
Without joining				
Total time	732s	4006s	35s	–
Solver time	555s	3312s	25s	–
Solver calls	6929	34944	66	–
Boolean simplification	82s	52s	3s	–
Paths for Symbolic Exec.	1662	256	65	2^{64}

Table 2: Results of applying state joining. Gzip with state joining timed out after 30,000 seconds.

With Boolean simplifications disabled, the example takes more than 50 times longer to run. It is well suited to state joining. Note that we pass the function symbolic variables, not characters turned into numbers. Parsing the numbers would require effort comparable to that in the Number example.

7 Complications

There are some practical complications with state joining for executables. Linux has hundreds of system calls that can modify the program’s state. Our tool has symbolic versions of the semantics for only a few system calls, for the remainder we replay the system call’s results which Valgrind captured during the dynamic disassembly. Before replaying the result of a system call we check that the parameters are the same. Our assumption is that system calls that are called in the same order with the same input will produce the same results. This limits further the strength of the guarantee we extract. For example if a program would behave differently on different dates, we would not discover this, as the result of the system call that returns the date is not made symbolic.

Because we replay traces of the system calls, if inputs cause the program to make different sequences of system calls, the analysis will not have the appropriate system call to replay. One solution may be to split the state whenever the sequence of system calls changes, but we do not do this yet.

If two states have different system call traces then they might not be appropriate to join. At present we cannot analyse the program shown in Fig. 7, as the

```

1 if (a>0)
2     p = malloc(100000);
3 else
4     p = malloc(0);
5 *p = 0;

```

Fig. 7: A complication for state joining

memory assigned on each branch is different. If we allowed the memory mapping system calls to vary on branches then one state would have memory allocated that the other did not. States cannot be joined just when their next instructions are the same. There are other limitations, such as the allocated memory, and the open files being the same.

Using dynamic disassembly we cannot visit locations that are not reached at runtime. So we cannot analyse error handling code unless the error occurs at runtime. For example, if we wish to insert error return values from system calls, such as a file read failing, then we need to disassemble the error handling code. Currently we cannot introduce failures when performing a disassembly, so cannot explore the error handling code.

Over-zealous joining is detrimental. For small functions we do not want to join, as the cost of joining/splitting overwhelms the saving. If a function is called from different sites and contains few instructions, the joined states will run for a few instructions before splitting when the function returns to the different calls sites. We need heuristics to identify the savings from joining states.

Another limitation is that symbolic execution generally operates on a fixed input width, say 20 bits. Depending on the program structure, greater input lengths may be required to cause a particular failure. Symbolic execution builds a circuit that describes the program for some fixed length input. For some inputs this may be equivalent to checking each smaller length input, but usually not.

When choosing what to join, we do not consider the calls site. Consider a function which is called at the start and end of a program. Our analysis does not take the call site into consideration, so believes that calling the function could return to either the start or end of the program—when it does not.

8 Related Research

The path explosion problem of symbolic execution has been addressed by others. Kölbl and Pixley [9] investigate state joining of programs written in a subset of C++, and describe it well. The principal difference with our work is that we focus on analysing arbitrary binaries which can use dynamic memory and pointer arithmetic.

Godefroid [7] performs a compositional analysis—a more general form of memoization. When a function is called with symbolic expressions, the PC when

it returns and the effects of the function are stored. Each time the function is called the PC and the results are saved. When calls to the same function are made later, before analysing the function again, a check is made whether the prior PCs cover the current PC. This approach has the advantage that function summaries can be reused; we inline functions, analysing them each time they are called. So a compositional approach would make at most 2^{64} passes through the Multiply function (Fig. 1). We do not reuse the summary, so analyse Multiply in entirety each time it is called. State joining and compositional approaches are orthogonal, and could be combined. Performing a compositional analysis of machine code is more difficult than on source code because the inputs to functions are not as obvious. Babić et al. [2] implement a similar memoization approach for their Calysto tool.

Boonstoppel et al. [4] discard states that differ from other states only in locations that will not later be read. Consider a conditional output statement `if (guard) {printf("value"); }`. If both branches are taken and outputs are ignored, the states do not differ. One can be discarded, as the remainder of their paths will be the same. This approach requires the calculation of which locations will be read and written to in the remainder of the path. Deciding this statically for machine code is more difficult owing to the more complicated control flow transfers. Our approach is more general—allowing the joining of states that differ in variables, while not requiring the calculation of which variables may be read from or written to later. The calculation does allow discarding of symbolic expressions that will not be used later—a good way to conserve memory.

Incidental to deriving the program’s input-output function, we extract an accurate (partial) CFG from the binary code. The approach generates a safe under-approximation of the CFG using a flow sensitive analysis. A corresponding upper-bound of the CFG can be produced by abstract interpretation [8].

In calculating the input-output function, our tool converts control dependency into data dependency. To automatically vectorise loops during compilation, Allen et al. [1] use Boolean simplification and *if-conversion* to similarly convert if-then-else statements. A statement such as `if(g) {y=x} else {y=z}` is converted into `y= (x & guard || y & guard)`, where *guard* is all ones or all zeroes, extended to the bit-width of *y*. We suspect that compilers, like gcc, which implement if-conversion will produce binaries that our tool can analyse more efficiently.

9 Discussion

State joining as we have implemented it, has varying performance. The performance of the approach depends on the difficulty of solving the generated constraints. On the gzip example, the constraints became so expensive to solve that state joining was slower both than executing the program and symbolic execution. In particular, gzip produced many symbolic memory indexes which slowed down the solvers.

State joining is useful if the following conditions apply: the paths call a similar sequence of system calls, the number of paths through the program is large, and memory is rarely written to at symbolic locations.

Three improvements to our implementation are apparent. First is that it is common around loops for later constraints to imply earlier ones. It is not apparent to a propositional simplifier that (from the Multiply example): $x_0 >_u 2 \neq 0$ implies $x_0 >_u 1 \neq 0$. Removing earlier constraints when they are implied by later constraints is very desirable. Second, and related, is that with our current simplification scheme based on propositional variables, the performance of the analysis is dependent on whether the constraints simplify during joining. If the joined PC can be simplified, as in the Multiply example, performance is good. However, slight syntactic changes to conditionals reduce the simplifications, dramatically increasing running time. Using the solver's native interface to maintain the state's PC would reduce the amount of work the solver needs to perform. Third, we plan to use the generalised memoization (compositional) approach to reduce the amount of re-work performed.

Normal symbolic execution of binaries allows arbitrary properties about the input-output function of programs to be verified, but the technique works poorly on programs that have many paths through them. We have investigated how state joining may help. So far we have a number of promising results for analysing unmodified executables, as well as examples that do not benefit. Future work will chip away at the latter.

References

1. J. R. Allen, K. Kennedy, C. Porterfield, and J. Warren. Conversion of control dependence to data dependence. In *Proceedings of the Tenth ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages*, pages 177–189. ACM Press, 1983.
2. D. Babić and A. Hu. Calysto: Scalable and precise extended static checking. In *Proceedings of the Thirtieth International Conference on Software Engineering*, pages 211–220. ACM Press, 2008.
3. G. Balakrishnan. *WYSINWYX: What You See Is Not What You Execute*. PhD thesis, University of Wisconsin at Madison, Madison, WI, USA, 2007.
4. P. Boonstoppel, C. Cadar, and D. R. Engler. RWset: Attacking path explosion in constraint-based test generation. In C. R. Ramakrishnan and J. Rehof, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, volume 4963 of *Lecture Notes in Computer Science*, pages 351–366. Springer, 2008.
5. C. Cadar, V. Ganesh, P. M. Pawlowski, D. L. Dill, and D. R. Engler. EXE: Automatically generating inputs of death. In *Proceedings of the Thirteenth ACM Conference on Computer and Communications Security*, pages 322–335. ACM Press, 2006.
6. V. Ganesh and D. L. Dill. A decision procedure for bit-vectors and arrays. In W. Damm and H. Hermanns, editors, *Computer Aided Verification*, volume 4590 of *Lecture Notes in Computer Science*, pages 524–536. Springer, 2007.
7. P. Godefroid. Compositional dynamic test generation. In *Proceedings of the Thirtieth ACM Symposium on Principles of Programming Languages*, pages 47–54. ACM Press, 2007.

8. J. Kinder, F. Zuleger, and H. Veith. An abstract interpretation-based framework for control flow reconstruction from binaries. In N. D. Jones and M. Müller-Olm, editors, *Proceedings of the Tenth International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI 2009)*, volume 5403 of *Lecture Notes in Computer Science*, pages 214–228. Springer, 2009.
9. A. Kölbl and C. Pixley. Constructing efficient formal models from high-level descriptions using symbolic simulation. *International Journal on Parallel Programming*, 33(6):645–666, 2005.
10. S.-I. Minato. Generation of BDDs from hardware algorithm descriptions. In *Proceedings of the 1996 IEEE/ACM International Conference on Computer-Aided Design*, pages 644–649. IEEE Comp. Soc., 1996.
11. S. Nanda, W. Li, L.-C. Lam, and T.-C. Chiueh. BIRD: Binary interpretation using runtime disassembly. In *Proceedings of the International Symposium on Code Generation and Optimization*, pages 358–370. IEEE Comp. Soc., 2006.
12. N. Nethercote and J. Seward. Valgrind: A framework for heavyweight dynamic binary instrumentation. In *Proceedings of the ACM SIGPLAN 2007 Conference on Programming Language Design and Implementation*, pages 89–100. ACM Press, 2007.
13. J. Patterson. Accurate static branch prediction by value range propagation. In *Proceedings of the ACM SIGPLAN 1995 Conference on Programming Language Design and Implementation*, pages 67–78. ACM Press, 1995.
14. R. L. Rudell. Multiple-valued logic minimization for PLA synthesis. Technical Report UCB/ERL M86/65, EECS Department, Berkeley, 1986.